Digital Signal Processing – Assignment 2
FIR Filters, Adaptive LMS Filtering, and Realtime ECG Peak Detection

Saad Aziz (GUID: 2917444A)
William Chambers (GUID: 2906463C)

University of Glasgow, School of Engineering

November 9, 2026

1 Recordings

[State the lab recording setup: two ECG recordings saved as `ecg1.tsv` (primary recording analysed across Tasks 1–5, chosen for its clear P-wave visibility) and `ecg2.tsv` (partner's recording used exclusively in Task 5 to extract the QRS matched-filter template). Note that the signal is read from ADC channel 2 (column 8, 0-indexed) and the sampling rate is $f_s = 1/(t_1 - t_0) = 250\,\text{Hz}$ (Nyquist frequency $f_s/2 = 125\,\text{Hz}$). Reminder: only quote numbers the submitted code prints or plots ($M = 1000$ taps, group delay 1.998 s, and 45 heart rates at a mean of 92.0 bpm); argue all other values visually from the figures.]

2 Task 1: Time and Frequency Analysis

[Briefly state how the time vector $t = n/f_s$ and single-sided linear magnitude spectrum $|\text{FFT}(\text{ecg})|/N$ up to $f_s/2$ were computed in `ecg_plot.py` without using prohibited high-level functions such as `fftfreq`. Emphasise that the spectrum is plotted on a **linear axis** as required by the brief.]

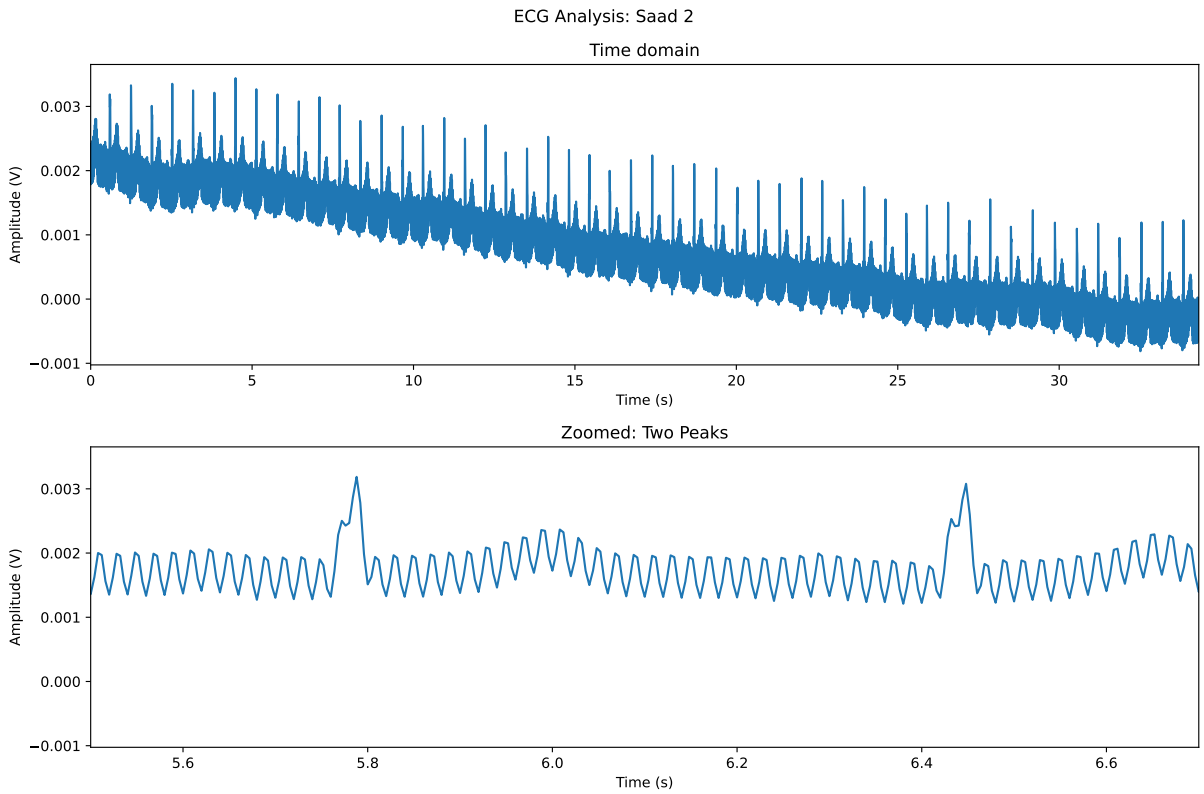


Figure 1: Raw primary ECG (`ecg1.tsv`) in the time domain: (top) full recording showing downward baseline drift, and (bottom) zoomed window (5.5 ˘ 6.7 s) spanning two heartbeats with visible PQRST complexes and 50 Hz ripple. [Annotate P, Q, R, S, T peaks in Inkscape if required.]

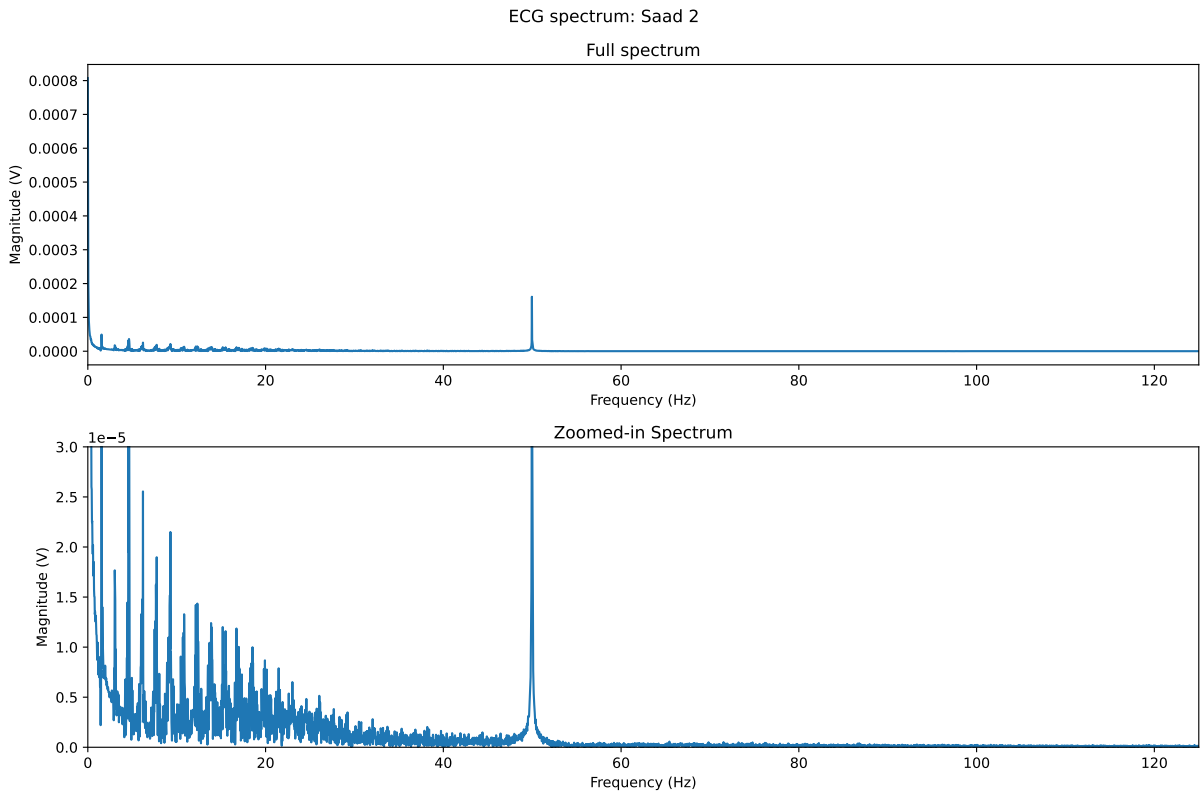


Figure 2: Single-sided linear frequency spectrum of `ecg1.tsv`: (top) full magnitude scale dominated by the 0 Hz DC/baseline spike and 50 Hz mains interference, and (bottom) zoomed magnitude scale ($0 - 3 \times 10^{-5}$ V) revealing the fundamental heart-rate peak and its harmonic comb.

ECG signal and harmonics. [From Fig. 1 (bottom), point out the two heartbeats in the 5.5 ˘ 6.7 s window (period ≈ 0.66 s, fundamental heart-rate frequency ≈ 1.5 Hz). In Fig. 2 (bottom), identify the comb of equally spaced ECG harmonics at multiples of the heart rate that gradually fade out by about 40 Hz.]

Noise and interferences. [Identify each interference source from Figs. 1–2: (1) **DC offset and baseline wander:** the whole time trace drifts downwards over the recording (while the R-peaks keep the same height above the local baseline, proving it is baseline wander rather than signal attenuation), producing the large spike at 0 Hz and the lowest frequency bins (< 0.5 Hz). Explicitly explain that **the DC bin is kept on purpose** because the 0 Hz spike represents the ADC offset plus baseline wander that our filter must remove—which is why a second, vertically zoomed spectrum is plotted; (2) **50 Hz mains interference:** visible as a regular sinusoidal ripple riding on the baseline in Fig. 1 (bottom) and as a sharp peak at 50 Hz in Fig. 2. Explicitly state that **no 100 Hz harmonic is visible** on the zoomed spectrum below the 125 Hz Nyquist frequency; (3) **Broadband EMG / muscle noise:** any broadband muscle noise overlaps the 0.5 ˘ 40 Hz ECG harmonic band and therefore cannot be removed by frequency-selective filtering without distorting the heartbeat itself.]

Frequencies to remove. [State and justify the two stopbands chosen for Task 2: (1) 0 ˘ 0.5 Hz to remove the DC offset and slow respiration/movement baseline wander while staying well below the ≈ 1.5 Hz fundamental heart rate; (2) 45 ˘ 125 Hz to remove the 50 Hz mains interference and all high-frequency noise up to Nyquist. Justify **why 45 Hz was chosen**: in Fig. 2 (bottom), the ECG harmonics fade by about 40 Hz and the 50 Hz interference skirt begins at about 48 Hz, so placing the cut-off at 45 Hz centres the filter's transition band in that natural spectral gap.]

3 Task 2: FIR Filter Coefficient Design

Number of coefficients. [Explain how `fir_design(sample_rate, ranges)` in `fir_design.py` automatically determines the tap count $M = 1000$: (1) It finds the narrowest stopband width $\Delta f_{\min} = \min(\text{high} - \text{low})$, which is $0 \text{ ˘ } 0.5\,\text{Hz}$ ($\Delta f_{\min} = 0.5\,\text{Hz}$); (2) It halves this width to set the frequency bin spacing $\delta f = \Delta f_{\min}/2 = 0.25\,\text{Hz}$ so that even the narrowest stopband spans at least two frequency bins; (3) It computes $M = \lfloor f_s/\delta f \rfloor = 250/0.25 = 1000$, and enforces even parity (`M += M % 2`) so the impulse response splits into two equal halves of length $M/2$ during the causal shift; (4) Discuss the delay trade-off: a linear-phase FIR filter has a group delay of $(M-1)/2$ samples $= 1.998\,\text{s}$. For a fixed M, increasing f_s reduces the delay; however, for a fixed frequency resolution δf, the delay $\approx (M-1)/(2f_s) \approx 1/(2\,\delta f)$ is fundamentally set by how close to DC (0.5 Hz) the filter must cut.]

IFFT coefficient calculation and edge cases. [Walk through the design steps in `fir_design.py`: (1) **Folded frequency axis and edge cases:** the bin frequencies $f_k = k\,f_s/M$ are folded via `f = np.minimum(f, sample_rate - f)` so bin $M-k$ is assigned the same frequency as bin k. This guarantees conjugate symmetry and handles the **DC and Nyquist edge cases** automatically: bin 0 is labelled 0 Hz (DC) and bin $M/2$ is labelled $f_s/2 = 125\,\text{Hz}$ (Nyquist); (2) **Ideal response:** $H[k]$ is initialised to 1 everywhere (passband) and set to 0 for every bin falling within the target stopband ranges ($0 \text{ ˘ } 0.5\,\text{Hz}$ and $45 \text{ ˘ } 125\,\text{Hz}$); (3) **IFFT and half-swap:** applying `np.real(ifft(H))` yields the zero-phase impulse response (the imaginary part vanishes because H is symmetric). Swapping the first and second halves (`h_shift`) centres the impulse response at $M/2$ to make the filter causal; (4) **Windowing:** multiplying by a Hamming window (`np.hamming(M)`) tapers the truncated impulse response tails to suppress Gibbs phenomenon passband/stopband ringing.]

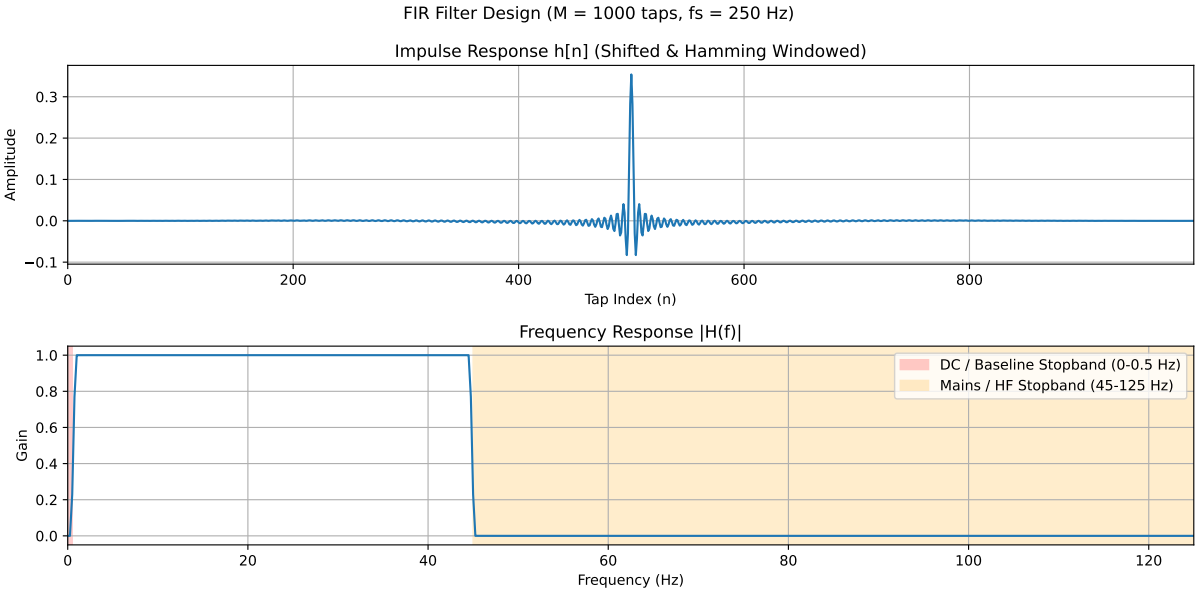


Figure 3: Designed FIR filter ($M = 1000$ taps, $f_s = 250\,\text{Hz}$): (top) shifted and Hamming-windowed impulse response $h[n]$ centred at $M/2 = 500$, and (bottom) frequency response magnitude $|H(f)|$ showing the $0 \text{ ˘ } 0.5\,\text{Hz}$ and $45 \text{ ˘ } 125\,\text{Hz}$ stopbands.

4 Task 3: Realtime FIR Filtering

Ring-buffer implementation. [Explain `class FIRfilter` in `fir_filter.py`: (1) `__init__(self, _coefficients)` stores the M tap weights, allocates a zero-initialised delay buffer of length

M, and sets the write pointer `self.offset = 0`; (2) `dofilter(self, v)` takes a single scalar input sample $x[n]$ and returns a single scalar output $y[n] = \sum_{m=0}^{M-1} h[m]\, x[n-m]$; (3) Emphasise that **a circular ring buffer is used instead of `np.roll` or array shifting**: nothing in the buffer is shifted in memory (which the brief penalises). Only the write pointer `self.offset` moves forward (wrapping to 0 at $M-1$), and the convolution sum is vectorised as two sliced dot products across the wrap-around boundary (`coefficients[:k]` with `buffer[offset::-1]` and `coefficients[k:]` with `buffer[:offset:-1]`).]

Filtered ECG results. [Discuss the realtime simulation in `fir_realtime_sim.py` (Fig. 4): (1) **Start-up transient:** the first $M = 1000$ samples (≈ 4 s) exhibit a start-up transient while the zero-initialised ring buffer fills with the DC-offset ECG signal, after which the baseline sits flat on 0 V; (2) **Delay-aligned PQRST verification:** after aligning the filtered trace by the $(M-1)/2 = 1.998$ s group delay (labelled 2.0 s in the plot legend), the zoomed comparison in Fig. 4 (middle) confirms that the 50 Hz ripple is completely eliminated while the P, Q, R, S, and T waves remain intact and undistorted; (3) **Spectral verification:** the filtered spectrum (bottom) confirms complete attenuation below 0.5 Hz and above 45 Hz while preserving the ECG harmonic comb.]

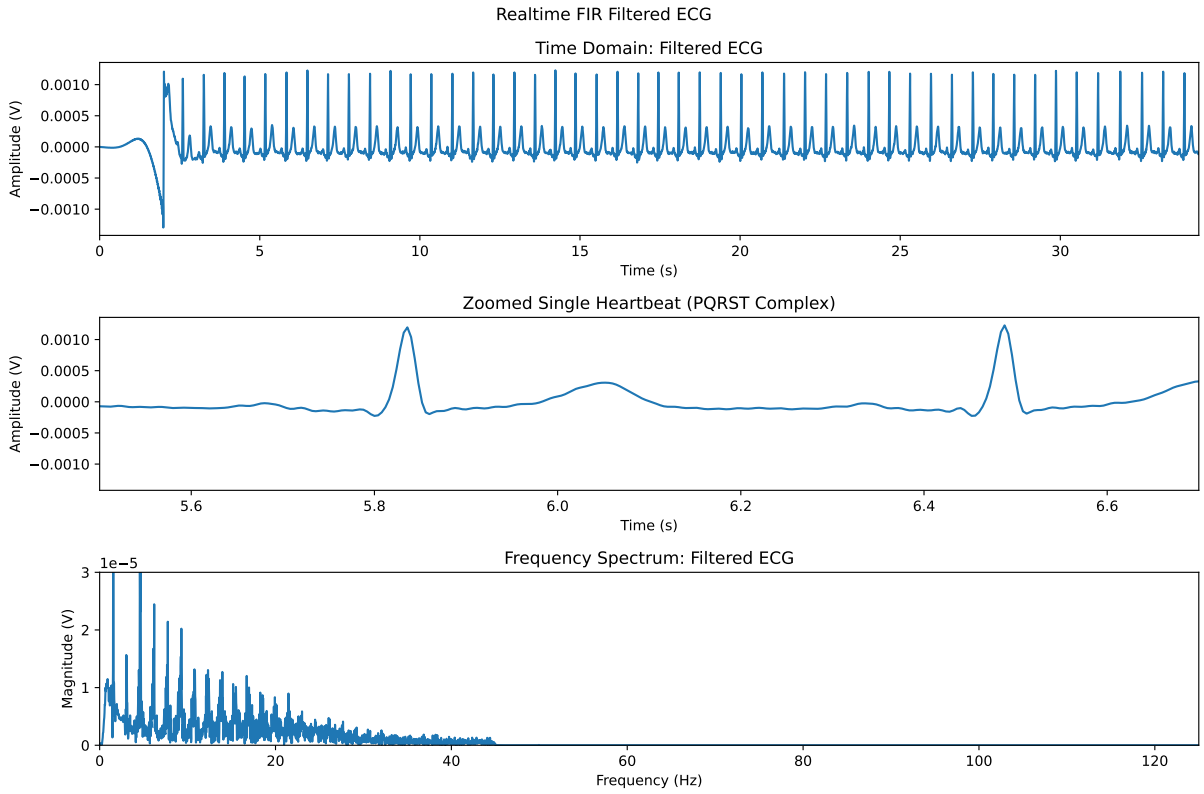


Figure 4: Task 3 sample-by-sample FIR filtering of `ecg1.tsv` ($M = 1000$, group delay 1.998 s): (top) full time-domain trace, (middle) zoomed heartbeat window (5.5 ˘ 6.7 s) verifying intact PQRST morphology, and (bottom) linear frequency spectrum.

5 Task 4: Adaptive LMS Filter

Implementation and series architecture. [Explain `doFilterAdaptive(self, signal, noise, learningRate)` in `fir_filter.py` and the two-stage design in `lms_filter.py`: (1) **Method operation:** on each scalar step, `doFilterAdaptive` saves the current write pointer `read_idx = self.offset`, passes the scalar reference `noise` $x[n]$ through `self.dofilter(noise)`

to form the interference estimate $y[n]$, computes the cleaned ECG error $e[n] = \text{signal} - y[n]$, and updates the tap weights in-place via $w_m \leftarrow w_m + \mu\, e[n]\, x[n-m]$ using the same two-slice ring-buffer indexing; (2) **Two cancellers in series:** a 1-tap DC/baseline canceller (constant reference 1.0, $\mu_{\text{dc}} = 0.0125$) is placed in series before a 10-tap 50 Hz canceller (sine reference $\sin(2\pi \cdot 50\, i/f_s)$, $\mu_{\text{lms}} = 0.01$, which requires multiple taps to adjust both amplitude and phase of the 50 Hz sinusoid); (3) **Qualitative justification of the design change:** state that a single LMS filter with a combined reference $(1.0 + \sin(\dots))$ was tried first, but its DC term adapted too quickly and acted as a high-pass filter that visibly flattened the slow P and T waves. Splitting into two series cancellers decouples the slow baseline adaptation from the fast 50 Hz cancellation.]

Comparison with the Task 3 FIR filter. [Compare the adaptive LMS output against the Task 3 fixed FIR output using Fig. 5: (1) **Waveform overlap:** once the Task 3 FIR trace is shifted back by its 1.998 s delay, the zoomed heartbeats overlap closely with intact PQRST waves; (2) **Frequency-domain difference:** the LMS filter acts as a narrow notch at DC and 50 Hz—it retains frequency content above 45 Hz and leaves a small 50 Hz spectral residual—whereas the Task 3 FIR filter removes everything from 45 ˘ 125 Hz, so the two filters do not perform quite the same job; (3) **Delay and settling:** the LMS filter has **no 2 s group delay** (only the FIR trace requires delay alignment) and uses only $1 + 10 = 11$ taps versus $M = 1000$ taps, while the DC canceller visibly settles to 0 V over the first few seconds of the recording.]

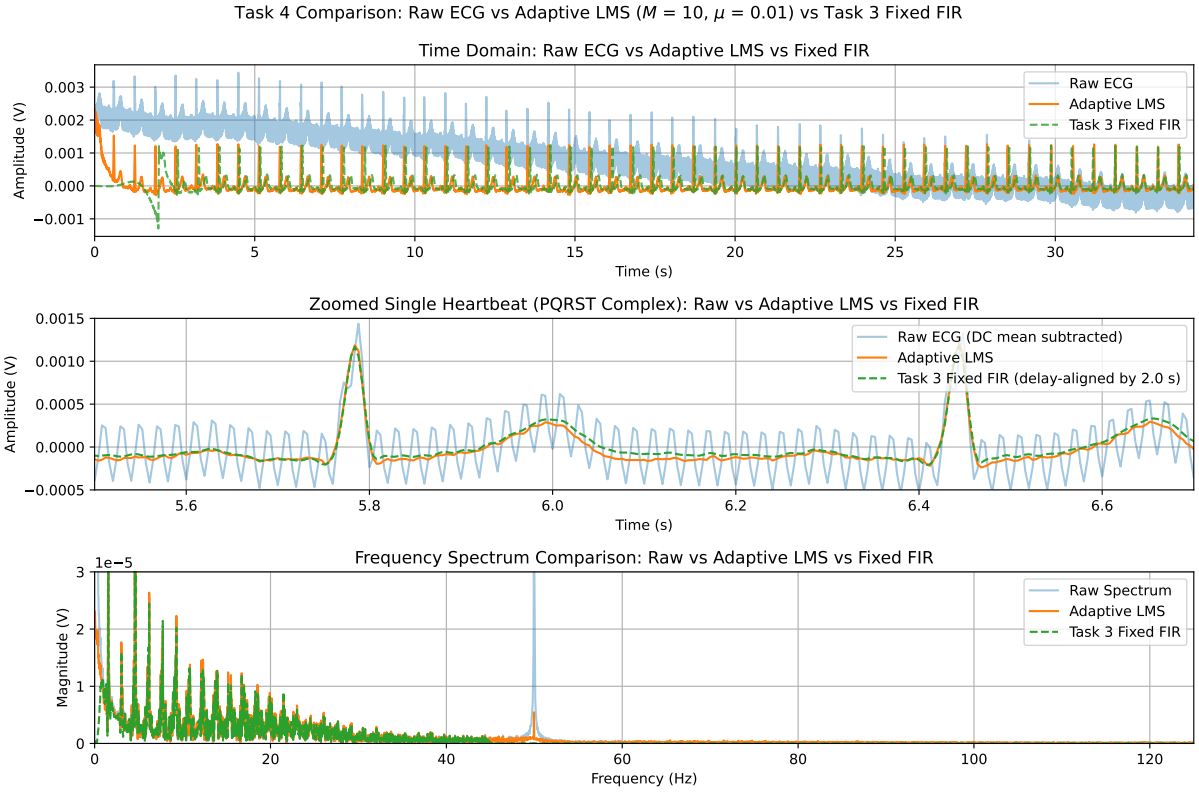


Figure 5: Comparison between the Task 3 fixed FIR filter ($M = 1000$) and the Task 4 two-stage adaptive LMS filter ($M_{\text{DC}} = 1, \mu_{\text{dc}} = 0.0125$; $M_{\text{LMS}} = 10, \mu_{\text{lms}} = 0.01$) across (top) full time domain, (middle) delay-aligned zoomed heartbeat, and (bottom) linear frequency spectrum.

6 Task 5: ECG Heartbeat Detection

Template extraction and matched filter. [Explain the matched filter design in `heartbeat.py`: (1) **Template source:** the partner's recording (`ecg2.tsv`) is first cleaned sample-by-sample using

the exact same Task 2 FIR filter (h) so that both the template and the test ECG have the same filtered shape. A 25-sample QRS segment (± 12 samples around $R = 2576$, i.e. ± 48 ms) is sliced *after* the cleaning filter's 1000-sample buffer has completely filled (Fig. 6, top); (2) **Time reversal:** the matched filter coefficients are set to the time-reversed template (`np.flip(template)`), because an FIR filter multiplies its first coefficient with the newest input sample (convolution), which equals cross-correlation when the template is reversed in time—avoiding prohibited functions like `correl` or `conv`; (3) **Squaring:** the matched filter output is squared (`** 2`) at each sample so that it is strictly positive and the tall R-peak correlation spikes are strongly emphasised relative to smaller Q/S/T side lobes.]

Thresholding, warm-up, and physiological heuristics. [Explain the realtime R-peak detector and heuristics in `heartbeat.py` (Fig. 6, bottom): (1) **Warm-up transient exclusion:** the first `TRANSIENTS` $= \text{len}(h) + \text{len}(\text{template}) = 1000 + 25 = 1025$ samples (≈ 4.1 s, shaded grey) are ignored because both cascaded filters in series (cleaning FIR + matched FIR) must fill their ring buffers before producing valid outputs; (2) **Threshold justification (1.5×10^{-11} V^4):** chosen directly from Fig. 6 (bottom), sitting with wide margin below all true R-peak spikes ($\geq 2.1 \times 10^{-11}$ V^4) and well above the secondary side spikes ($\approx 0.8 \times 10^{-11}$ V^4); (3) **Rising-edge detection:** a beat is triggered only when the squared output crosses upward through the threshold (`out[i] > THRESHOLD and out[i-1] <= THRESHOLD`), ensuring each peak is counted once; (4) **Heuristics:** two rules filter the momentary heart rate HR $= 60\, f_s/(n_{\text{peak}} - n_{\text{last}})$: first, physiological bounds of 30 ˘ 200 bpm (established from discussion with a physiology student); second, a beat-to-beat consistency check requiring each new rate to fall within $\pm 20\%$ (0.80 ˘ 1.20×) of the previous accepted rate, which automatically rejects missed beats (-50% apparent rate) and double detections ($+100\%$ apparent rate).]

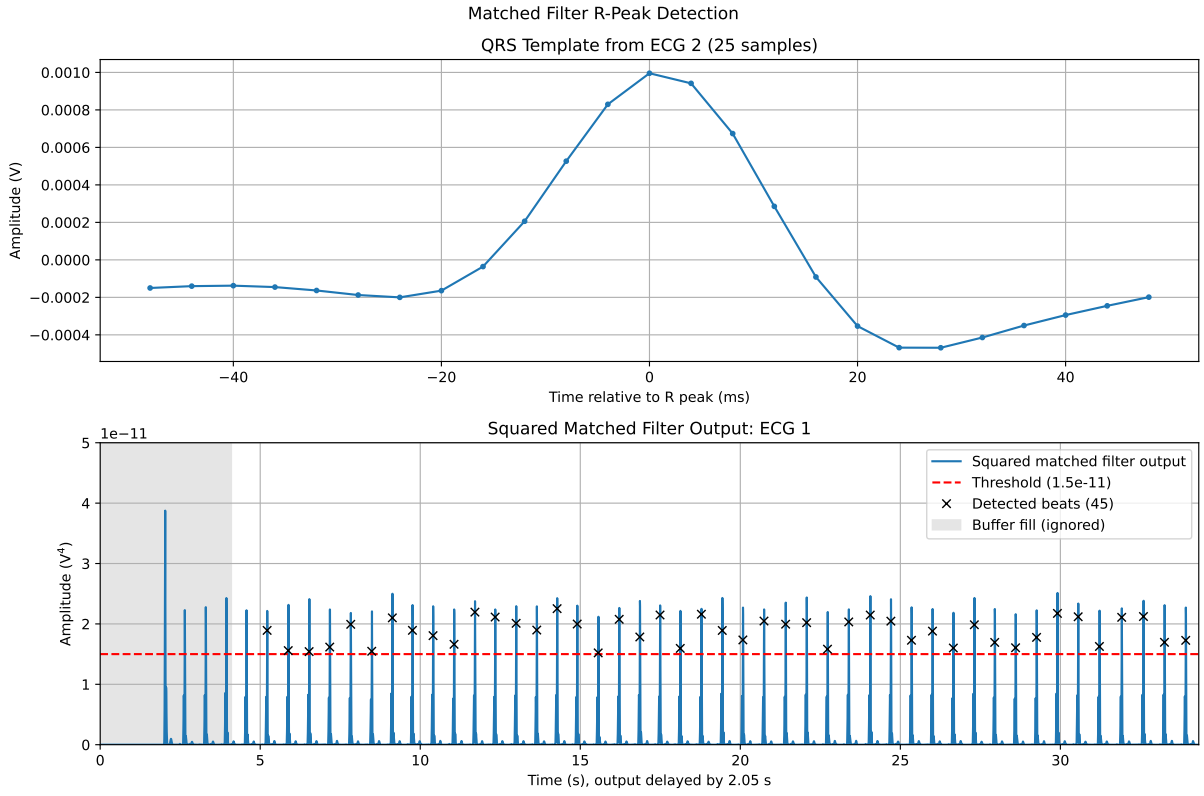


Figure 6: Task 5 matched-filter R-peak detection: (top) 25-sample QRS template extracted from cleaned `ecg2.tsv` around sample $R = 2576$, and (bottom) squared matched-filter output on `ecg1.tsv` showing the 1025 sample buffer-fill exclusion zone, the 1.5×10^{-11} V^4 threshold, and the 45 accepted beat intervals.

Momentary heart rate analysis. [Discuss the momentary heart-rate plot in Fig. 7: (1) **46 beats → 45 heart rates:** the detector finds 46 valid R-peaks after the warm-up period, yielding 45 inter-beat intervals and a printed mean heart rate of 92.0 bpm (the first detected beat has no preceding beat to form an interval, so it produces no rate and is not plotted); (2) **Heart-rate variability (HRV):** the momentary heart rate fluctuates naturally by a few bpm from beat to beat around 92.0 bpm. Explain that this is normal physiological heart-rate variability (respiratory sinus arrhythmia, where heart rate rises slightly during inhalation and falls during exhalation) rather than detection jitter; (3) **Effect of filter delay:** the combined linear-phase delay of the cleaning filter (1.998 s) and matched filter (0.048 s) is 2.046 s (displayed as 2.05 s). Because every detected beat is delayed by the exact same constant offset, the time intervals between beats—and hence the momentary heart rates—are completely unaffected (and the plotted time axis is shifted back by `delay` to align with the raw ECG).]

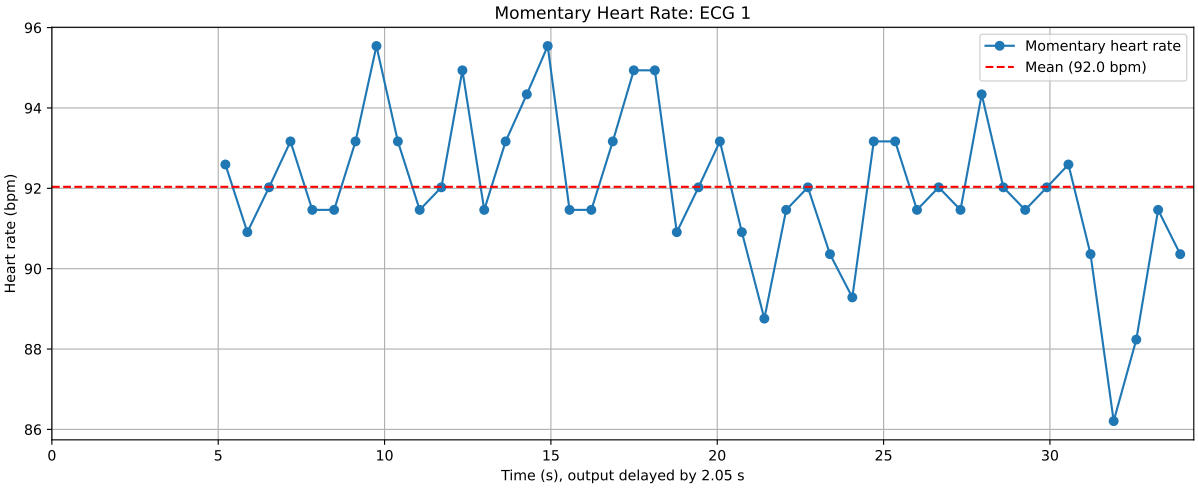


Figure 7: Momentary heart rate over time for `ecg1.tsv` across 45 beat-to-beat intervals (mean heart rate 92.0 bpm), delay-aligned to the input ECG timeline.

A Acknowledgement of GenAI Use

Tools: [State exact tool name, version, and publisher, e.g. Claude Opus 4.5 / 5.5 (Anthropic).]
Use: [State how GenAI was used (e.g. suggesting the folded frequency axis `np.minimum(f, sample_rate - f)` in Task 2 and assisting with matplotlib formatting for the momentary heart-rate plot in Task 5) and what work the team contributed directly (lab ECG recordings, spectral diagnosis and cut-off selection, tap-count derivation, ring-buffer FIR and two-stage series LMS implementation, matched-filter template extraction and heuristic design, and writing the report).]

B Usage Instructions

All scripts use platform-independent relative paths (`pathlib.Path`) and require `numpy`, `scipy`, and `matplotlib`. After unzipping the submission archive, run each script directly from a terminal in the unzipped directory:

```
python ecg_plot.py
python fir_realtime_sim.py
python lms_filter.py
python heartbeat.py
```

C Python Code

C.1 support_funcs.py

```python
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR = Path(__file__).resolve().parent


def import_ecg_data(filename):
    """
    Parameters:
    filename: str or Path
        The name of the file containing the ECG data.

    Returns:
    tuple[np.ndarray, np.ndarray, float]
        (t, ecg, sample_rate) - time vector (s), ECG signal (V), and sampling rate (Hz).
    """
    ECG_COLUMN = 8  # ADC channel 2
    data = np.loadtxt(BASE_DIR / filename)
    t = data[:, 0]
    ecg = data[:, ECG_COLUMN]
    sample_rate = 1 / (t[1] - t[0])  # fs = 250 Hz

    return t, ecg, sample_rate
```

C.2 ecg_plot.py

```python
# Task 1

import numpy as np
from scipy.fft import fft
import matplotlib.pyplot as plt

from support_funcs import import_ecg_data
```

```
8
9   # Task 1 Plots
10  # Function to plot ECG signal in the time and frequency domain
11  def plot_ecg(name, sample_rate, ecg, beat_start, beat_end):
12      n = len(ecg)
13      time = np.arange(n) / sample_rate # array of times
14
15      # Frequency domain (linear)
16      ecg_fft = (1 / n) * fft(ecg)
17      mag_fft = np.abs(ecg_fft)
18      freqs = np.arange(n // 2) * sample_rate / n # frequency bins up to sample_rate/2
19
20      plt.figure(figsize=(12, 8))
21      plt.suptitle(f"ECG Analysis: {name}")
22
23      # fig 1: time domain
24      plt.subplot(2, 1, 1)
25      plt.title("Time domain")
26      plt.plot(time, ecg)
27      plt.xlim(0, n / sample_rate)
28      plt.xlabel("Time (s)")
29      plt.ylabel("Amplitude (V)")
30
31      # zoomed in
32      plt.subplot(2, 1, 2)
33      plt.title("Zoomed: two heartbeats")
34      plt.plot(time, ecg)
35      plt.xlim(beat_start, beat_end)
36      plt.xlabel("Time (s)")
37      plt.ylabel("Amplitude (V)")
38
39      plt.tight_layout() # stop axes overlapping
40
41      # fig 2: frequency domain
42      plt.figure(figsize=(12, 8))
43      plt.suptitle(f"ECG spectrum: {name}")
44
45      plt.subplot(2, 1, 1)
46      plt.title("Full spectrum")
47      plt.plot(freqs[0:], mag_fft[0:n // 2])
48      plt.xlim(0, sample_rate / 2)
49      plt.xlabel("Frequency (Hz)")
50      plt.ylabel("Magnitude (V)")
51
52      # zoomed in
53      plt.subplot(2, 1, 2)
54      plt.title("Zoomed-in Spectrum")
55      plt.plot(freqs[0:], mag_fft[0:n // 2])
56      plt.xlim(0, sample_rate / 2)
57      plt.ylim(0, 3e-5) # reveals the small peaks
58      plt.xlabel("Frequency (Hz)")
59      plt.ylabel("Magnitude (V)")
60
61      plt.tight_layout() # stop axes overlapping
62
63  # import ECG Data & plot
64  _, ecg, sample_rate = import_ecg_data("ecg1.tsv")
65  plot_ecg("ECG 1", sample_rate, ecg, 5.5, 6.7) # zoom window containing two beats
66  plt.show()
```

C.3 fir_design.py

```python
#Task 2
import numpy as np
from scipy.fft import ifft

#This function determines the FIR coefficients for freq. ranges specified in the function arguments (2D array: [low, high]) and a given sample rate.
def fir_design(sample_rate, ranges):

    ranges = np.array(ranges)
    min_width = np.min(ranges[:, 1] - ranges[:, 0])

    bin_size = min_width / 2
    M = int(sample_rate / bin_size) #half the narrowest stopband, so every stopband covers at least 2 bins

    M += M%2 # ensure M is even

    #Using folded axis to create a symmetric frequency response
    f = np.arange(M) * sample_rate / M #freq. of each bin
    f = np.minimum(f, sample_rate - f) #fold: bin M-k gets the same freq. as bin k
    H = np.ones(M) #initialise frequencies to 1 (pass band)

    #define low and high, setting the bins within this band to 0.
    for low, high in ranges:
        H[(f >= low) & (f <= high)] = 0 #remove bins in range (set to 0)

    # IFFT - produces the impulse response, keeping only the real part (H is mirrored, so complex cancels)
    h = np.real(ifft(H))

    # Swap the two halves: centres the impulse response at M/2
    h_shift = np.zeros(M)
    h_shift[0:M//2] = h[M//2:M]
    h_shift[M//2:M] = h[0:M//2]

    h_windowed = h_shift * np.hamming(M) #apply hamming window to reduce Gibbs ripple from truncation

    #Edge cases (DC and Nyquist) are covered by the folded axis (bin 0 and bin M/2 [fs/2])

    return h_windowed
```

C.4 fir_filter.py

```python
# task 3a + 4a - FIRfilter class functions
import numpy as np


class FIRfilter:
    def __init__(self, _coefficients):
        """
        Parameters:
        _coefficients: array
            The M filter coefficients calculated in Task 2.
        """
        self.coefficients = np.array(_coefficients, dtype=float) # load coefficients into object
        self.ntaps = len(self.coefficients) # get buffer array length

        self.buffer = np.zeros(self.ntaps, dtype=float) # generate buffer array

        self.offset = 0 # ring buffer index where the next sample is written

    def dofilter(self, v):
```

```
        """
        Parameters:
        v: float
            The incoming input sample x[n].

        Returns:
        float
            The filtered output sample y[n].
        """
        self.buffer[self.offset] = v # load current sample into buffer

        k = self.offset + 1
        # output = sum[0:M-1](h(m) \dot x(n-m))
        output = (
            np.dot(self.coefficients[:k], self.buffer[self.offset::-1])  # fwd sum
            + np.dot(self.coefficients[k:], self.buffer[:self.offset:-1]) # backward sum
        ) # dot product multiply coefficients with samples in object

        # iterate offset position, reset to zero
        if self.offset == self.ntaps - 1:
            self.offset = 0
        else:
            self.offset += 1


        return float(output)

    def doFilterAdaptive(self, signal, noise, learningRate):
        """
        Parameters:
        signal: float
            Current contaminated ECG sample d[n].
        noise: float
            Current reference noise sample x[n] (e.g. sin(2*pi*50/fs * n)).
        learningRate: float
            LMS step size (mu) controlling how fast the weights adapt.

        Returns:
        float
            The cleaned ECG sample e[n] = d[n] - y[n].
        """
        read_idx = self.offset # store current write index before dofilter increments offset

        y = self.dofilter(noise) # filter reference noise x[n] to estimate interference y[n]

        e = signal - y # subtract estimated noise from signal to get error / clean ECG e[n]

        k = read_idx + 1
        step = learningRate * e # calculate LMS weight update factor (mu * e[n])
        # w_m[n+1] = w_m[n] + mu * e[n] * x[n-m]
        self.coefficients[:k] += step * self.buffer[read_idx::-1] # fwd tap weight update
        self.coefficients[k:] += step * self.buffer[:read_idx:-1] # backward tap weight update

        return float(e)
```

C.5 fir_realtime_sim.py

```
# Task 3b - Simulating realtime FIR filtering importing fir_design and fir_filter.

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

```

```python
from fir_design import fir_design
from fir_filter import FIRfilter
from support_funcs import import_ecg_data

# FIR Filter params from Task 2 (also used by heartbeat.py)
BASELINE_WANDER = [0, 0.5]  # Remove DC and low-frequency baseline drift
MAINS_AND_HF_HUM = [45, 125]  # Remove 50 Hz mains hum and high-frequency noise up to Nyquist (fs/2)

def run_fir_filter(ecg, sample_rate):
    # generate list of tap coeffs
    h = fir_design(sample_rate, [BASELINE_WANDER, MAINS_AND_HF_HUM])

    # create object and generate output list
    fir = FIRfilter(h)
    ecg_filtered = []

    for sample in ecg:
        ecg_filtered.append(fir.dofilter(sample))

    # convert list into np to plot
    ecg_filtered = np.array(ecg_filtered)
    group_delay = (len(h) - 1) / 2 / sample_rate # linear-phase FIR delay: (M-1)/2 samples
    print(f"FIR filter: M = {len(h)} taps, group delay = {group_delay:.3f} s")

    return ecg_filtered, group_delay


if __name__ == "__main__": # since its being imported we need to prevent plotting repeats
    # load ECG Data
    t, ecg, sample_rate = import_ecg_data("ecg1.tsv")

    ecg_filtered, group_delay = run_fir_filter(ecg, sample_rate)
    n = len(ecg_filtered)

    # generate plots
    # Filtered ECG (Time Domain, Zoomed Heartbeat, Frequency Spectrum) ---
    freqs = np.arange(n // 2) * sample_rate / n
    raw_fft = (1 / n) * np.abs(fft(ecg))
    filt_fft = (1 / n) * np.abs(fft(ecg_filtered))

    beat_start, beat_end = 5.5, 6.7
    in_beat = (t >= beat_start) & (t <= beat_end)

    plt.figure(figsize=(12, 8))
    plt.suptitle("Task 3: Realtime FIR Filtered ECG: ECG 1")

    # Full time domain
    plt.subplot(3, 1, 1)
    plt.title("Time Domain: Raw vs Filtered ECG")
    plt.plot(t, ecg, alpha=0.5, label="Unfiltered ECG")
    plt.plot(t, ecg_filtered, label="Filtered ECG")
    plt.xlim(0, n / sample_rate)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")
    plt.legend(loc="upper right")
    plt.grid()

    # Zoomed single heartbeat (PQRST check)
    plt.subplot(3, 1, 2)
    plt.title("Zoomed Heartbeat (PQRST Complex): Raw vs Filtered ECG")
    plt.plot(t, ecg - np.mean(ecg[in_beat]), alpha=0.5, label="Unfiltered ECG (DC mean subtracted)")
```

```
    plt.plot(t - group_delay, ecg_filtered, label=f"Filtered ECG (delay-aligned by {group_delay
    :.1f} s)")
    plt.xlim(beat_start, beat_end)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")
    plt.legend(loc="lower right")
    plt.grid()

    # Linear frequency spectrum
    plt.subplot(3, 1, 3)
    plt.title("Frequency Spectrum: Raw vs Filtered ECG")
    plt.plot(freqs, raw_fft[:n // 2], alpha=0.5, label="Unfiltered Spectrum")
    plt.plot(freqs, filt_fft[:n // 2], label="Filtered Spectrum")
    plt.xlim(0, sample_rate / 2)
    plt.ylim(0, 3e-5)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")
    plt.legend(loc="upper right")
    plt.grid()

    plt.tight_layout()
    plt.show()
```

C.6 lms_filter.py

```
# Task 4b - Simulating realtime adaptive LMS filtering importing fir_filter.

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from fir_filter import FIRfilter
from fir_realtime_sim import run_fir_filter
from support_funcs import import_ecg_data

# Load ECG Data
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")

# Run Task 3 FIR filter for comparison
ecg_fir, group_delay = run_fir_filter(ecg, sample_rate)

# Adaptive LMS Filter Setup: two cancellers in series

# DC / baseline wander canceller (constant reference = 1.0)
M_DC = 1 # number of taps
mu_dc = 0.0125 # learning rate
lms_dc = FIRfilter(np.zeros(M_DC)) # adaptive filter, weights start at 0

# 50 Hz mains canceller (50 Hz sine reference)
M_lms = 10 # number of taps
mu_lms = 0.01 # learning rate
f_noise = 50.0 # ref signal frequency
lms = FIRfilter(np.zeros(M_lms)) # adaptive filter, weights start at 0

# Realtime Adaptive Filtering
ecg_lms = [] # results list

for i, sample in enumerate(ecg):

    # remove DC / baseline wander
    ecg_no_dc = lms_dc.doFilterAdaptive(sample, 1.0, mu_dc)
    # remove 50 Hz powerline interference
    noise_50 = np.sin(2 * np.pi * f_noise * i / sample_rate)
```

```
    ecg_lms.append(lms.doFilterAdaptive(ecg_no_dc, noise_50, mu_lms))

ecg_lms = np.array(ecg_lms) # convert to np array to plot
n = len(ecg_lms)

# Plotting (FIR Filtered ECG vs Adaptive LMS Filtered ECG)
freqs = np.arange(n // 2) * sample_rate / n
fir_fft = (1 / n) * np.abs(fft(ecg_fir))
lms_fft = (1 / n) * np.abs(fft(ecg_lms))

beat_start, beat_end = 5.5, 6.7

plt.figure(figsize=(12, 8))
plt.suptitle("Task 4: Realtime Adaptive LMS vs FIR Filtered ECG: ECG 1")

# Full time domain
plt.subplot(3, 1, 1)
plt.title("Time Domain: FIR vs Adaptive LMS Filtered ECG")
plt.plot(t, ecg_fir, alpha=0.7, label="FIR Filtered ECG")
plt.plot(t, ecg_lms, alpha=0.8, label="Adaptive LMS Filtered ECG")
plt.xlim(0, n / sample_rate)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid()

# Zoomed single heartbeat (PQRST check)
plt.subplot(3, 1, 2)
plt.title("Zoomed Heartbeat (PQRST Complex): FIR vs Adaptive LMS Filtered ECG")
plt.plot(t - group_delay, ecg_fir, alpha=0.8, label=f"FIR Filtered ECG (delay-aligned by {
    group_delay:.1f} s)")
plt.plot(t, ecg_lms, alpha=0.8, label="Adaptive LMS Filtered ECG")
plt.xlim(beat_start, beat_end)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="lower right")
plt.grid()

# Linear frequency spectrum
plt.subplot(3, 1, 3)
plt.title("Frequency Spectrum: FIR vs Adaptive LMS Filtered ECG")
plt.plot(freqs, fir_fft[:n // 2], alpha=0.7, label="FIR Spectrum")
plt.plot(freqs, lms_fft[:n // 2], alpha=0.8, label="Adaptive LMS Spectrum")
plt.xlim(0, sample_rate / 2)
plt.ylim(0, 3e-5)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (V)")
plt.legend(loc="upper right")
plt.grid()

plt.tight_layout()
plt.show()
```

C.7 heartbeat.py

```
#5 - simulating real time heartbeat detection using fir_filter

import numpy as np
import matplotlib.pyplot as plt

from fir_design import fir_design
from fir_filter import FIRfilter
from fir_realtime_sim import BASELINE_WANDER, MAINS_AND_HF_HUM
```

```
from support_funcs import import_ecg_data


# Load and format ECG data
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")  # ECG to analyse
_, template_ecg, _ = import_ecg_data("ecg2.tsv")  # other lab partner's ECG


#Apply filter from Task 3
h = fir_design(sample_rate, [BASELINE_WANDER, MAINS_AND_HF_HUM])

#Extract QRS from TEMPLATE
template_fir = FIRfilter(h)
template_clean = np.zeros(len(template_ecg))
for i in range(len(template_ecg)):
    template_clean[i] = template_fir.dofilter(template_ecg[i])


#Cut around the R peak (include QRS). R is chosen after the cleaning filter's buffer has filled
R = 2576 #sample number of R peak in template ECG (ecg2.tsv)
template = template_clean[R-12:R+13] #25 samples
matched_fir = FIRfilter(np.flip(template)) #reverse coefficients in time
ecg_fir = FIRfilter(h) #fresh cleaning filter for ECG

#matched filter output, stored for plotting
matched_fir_out = np.zeros(len(ecg))

# --- Realtime detection ---
THRESHOLD = 1.5e-11 #from matched filter plot
TRANSIENTS = len(template) + len(h) # ignore output until both filter buffers are full (cleaning
     + matched filter)
MIN_BPM = 30
MAX_BPM = 200
last_beat = None #index of previous beat
heart_rate = []
beat_idx = [] #sample index of each beat with an accepted rate

for i in range(len(ecg)):
    matched_fir_out[i] = matched_fir.dofilter(ecg_fir.dofilter(ecg[i])) ** 2 #square to make
    positive and emphasise peaks
    if i <= TRANSIENTS:
        continue
    if matched_fir_out[i] > THRESHOLD and matched_fir_out[i-1] <= THRESHOLD:
        peak = i
        if last_beat != None:
            #Ensure heartrate is within reasonable range (30-200 bpm)
            momentary_rate = 60* sample_rate / (peak - last_beat) #bpm
            in_range =  MIN_BPM < momentary_rate < MAX_BPM #Heuristic 1

            prev_rate = heart_rate[-1] if heart_rate else momentary_rate #prevent IndexError
            consistency = 0.80*prev_rate < momentary_rate < 1.20*prev_rate #Heuristic 2
            if in_range and consistency:
                heart_rate.append(momentary_rate)
                beat_idx.append(peak)
        last_beat = peak

print(f"Heart rates calculated: {len(heart_rate)}, mean heart rate: {np.mean(heart_rate):.1f}
    bpm")

# --- PLOTS ---
delay = ((len(h) - 1) / 2 + (len(template) - 1) / 2) / sample_rate # exact delay of cleaning +
    matched filter: (M-1)/2 samples each
```

```
template_ms = (np.arange(len(template)) - len(template) // 2) / sample_rate * 1000 # time relative to R peak

plt.figure(figsize=(12, 8))
plt.suptitle("Matched Filter R-Peak Detection")

# Template (matched filter coefficients before time reversal)
plt.subplot(2, 1, 1)
plt.title(f"QRS Template from ECG 2 ({len(template)} samples)")
plt.plot(template_ms, template, marker=".")
plt.xlabel("Time relative to R peak (ms)")
plt.ylabel("Amplitude (V)")
plt.grid(True)

# Squared matched filter output with threshold and detected beats
plt.subplot(2, 1, 2)
plt.title("Squared Matched Filter Output: ECG 1")
plt.plot(t, matched_fir_out, label="Squared matched filter output")
plt.axhline(THRESHOLD, color="red", linestyle="--", label=f"Threshold ({THRESHOLD:.1e})")
plt.plot(t[beat_idx], matched_fir_out[beat_idx], "x", color="black", label=f"Detected beats ({len(beat_idx)})")
plt.axvspan(0, TRANSIENTS / sample_rate, color="grey", alpha=0.2, label="Buffer fill (ignored)")
plt.xlim(0, len(ecg) / sample_rate)
plt.ylim(0, 0.5e-10)
plt.xlabel(f"Time (s), output delayed by {delay:.2f} s")
plt.ylabel("Amplitude (V$^4$)")
plt.legend(loc="upper right")
plt.grid(True)

plt.tight_layout()

# Momentary heart rate, shifted back by the filter delay so times match the ECG
plt.figure(figsize=(12, 4))
plt.title(f"Momentary Heart Rate: ECG 1 (mean {np.mean(heart_rate):.1f} bpm)")
plt.plot(t[beat_idx] - delay, heart_rate, marker=".")
plt.xlim(0, len(ecg) / sample_rate)
plt.xlabel("Time (s)")
plt.ylabel("Heart rate (bpm)")
plt.grid(True)

plt.tight_layout()

plt.show()
```